Сортировки

Что это и зачем надо

Алгоритмы сортировки — основа программирования, без них невозможно двигаться дальше. Но почему сортировка имеет такое значение? Дело в том, что **упорядочивание** — это основа для оптимизации других алгоритмов, улучшения производительности и более эффективного управления данными.

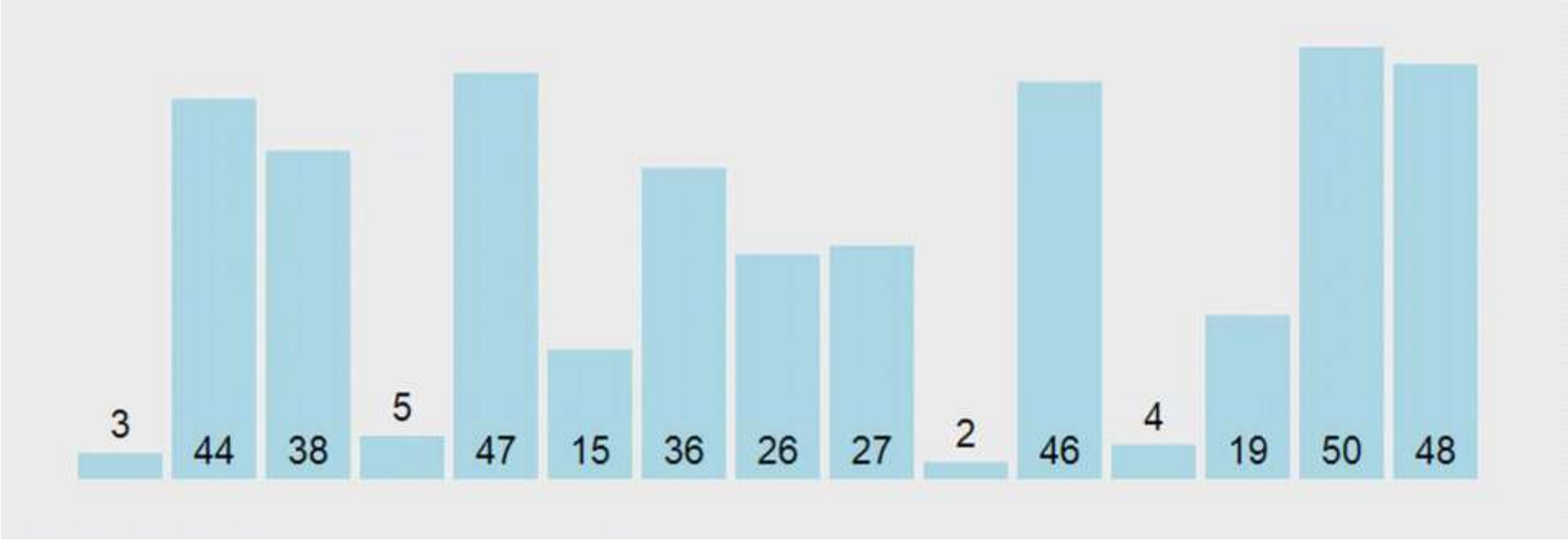
Что это и зачем надо

При этом, если представить сортировку просто как способ расположить элементы в нужном порядке, мы упустим ее глубокий смысл. Алгоритмы сортировки не просто «переставляют» элементы массива. Они оптимизируют работу приложений, минимизируют потребление памяти и могут работать даже на больших наборах данных

Оптимальные алгоритмы позволяют снизить время выполнения программ и обеспечить предсказуемое поведение системы. Без них приложения будут страдать от затяжных задержек или даже сбоев при работе с большими данными.

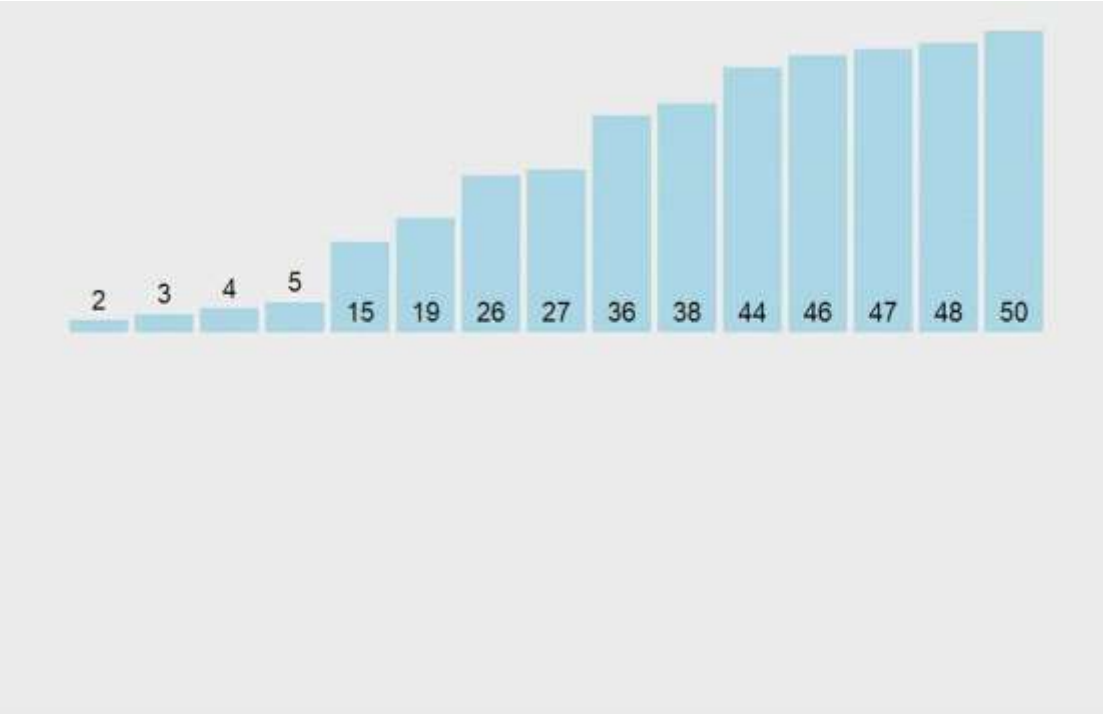
Популярные алгоритмы сортировки

Пузырьковая сортировка (Bubble Sort). Простейший алгоритм, который сравнивает пары соседних элементов и меняет их местами. Несмотря на свою простоту, это один из наименее эффективных алгоритмов со сложностью O(n2).



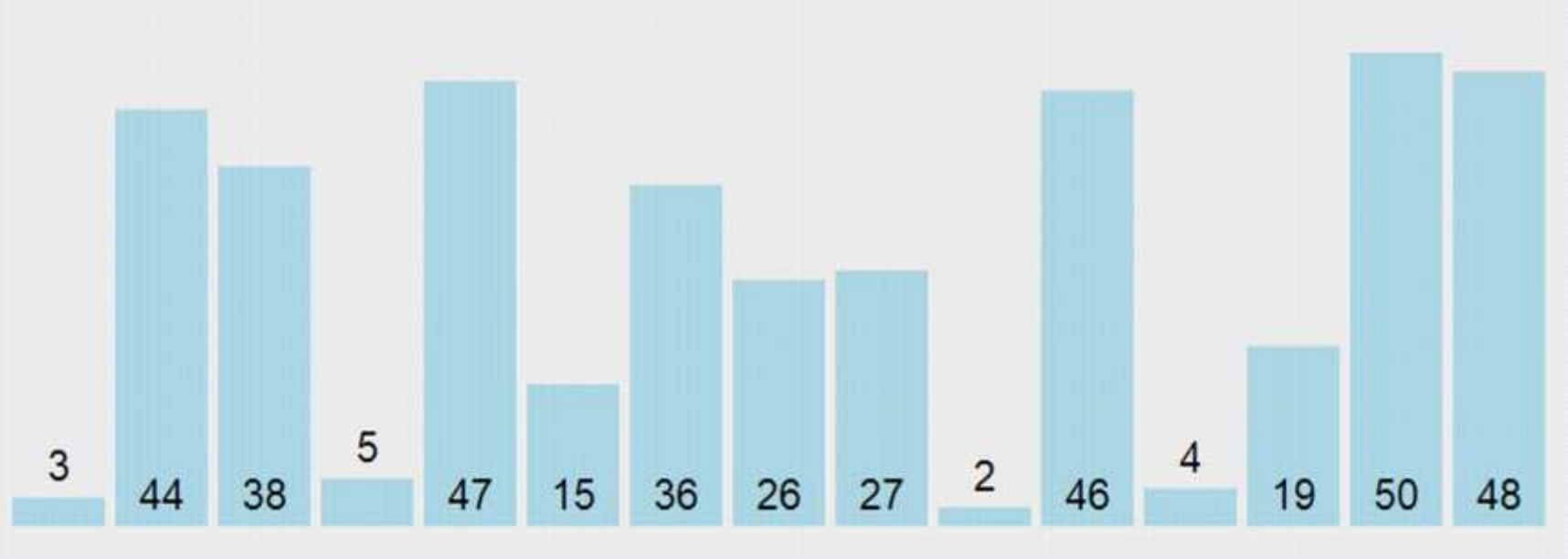
Популярные алгоритмы сортировки

Сортировка вставками (Insertion Sort). Работает путем вставки каждого нового элемента в уже отсортированный массив. Эффективен для небольших массивов и имеет временную сложность O(n2).



Популярные алгоритмы сортировки

Сортировка выбором (Selection Sort). На каждом шаге выбирает минимальный элемент из неотсортированной части и помещает его в конец отсортированной. Подходит для массивов, где важна простота реализации. Его эффективность также O(n2).



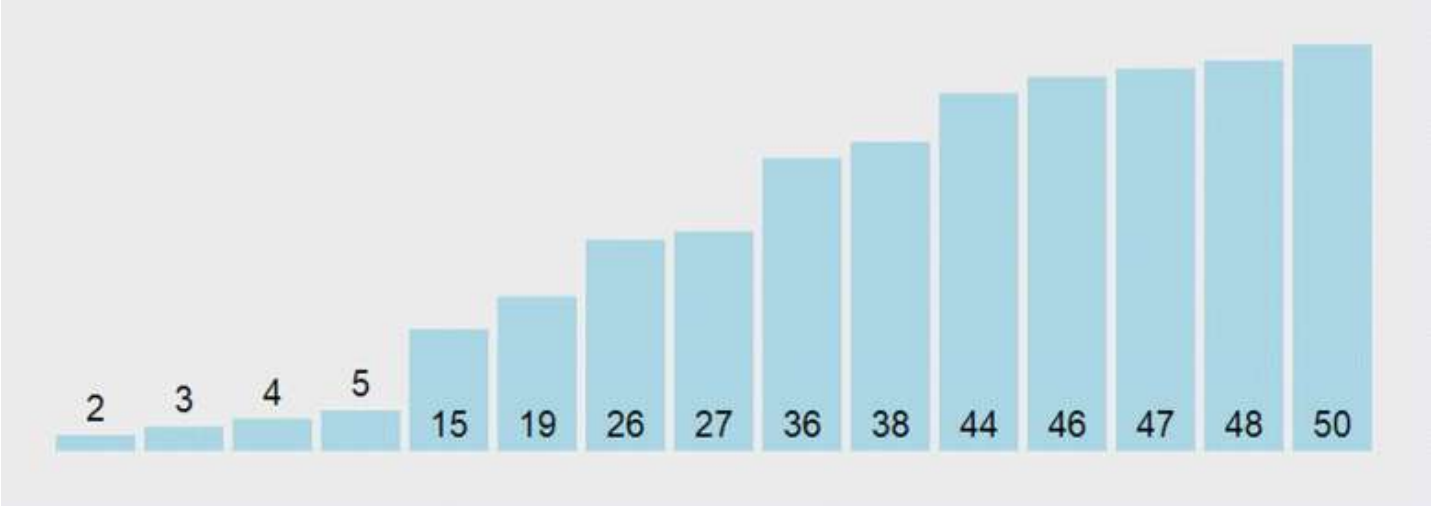
Популярные алгоритмы сортировки

Пирамидальная сортировка (Heap Sort). Использует структуру данных «куча» и работает за O(n log n). Отличается предсказуемой производительностью и малыми накладными расходами.

6 5 3 1 8 7 2 4

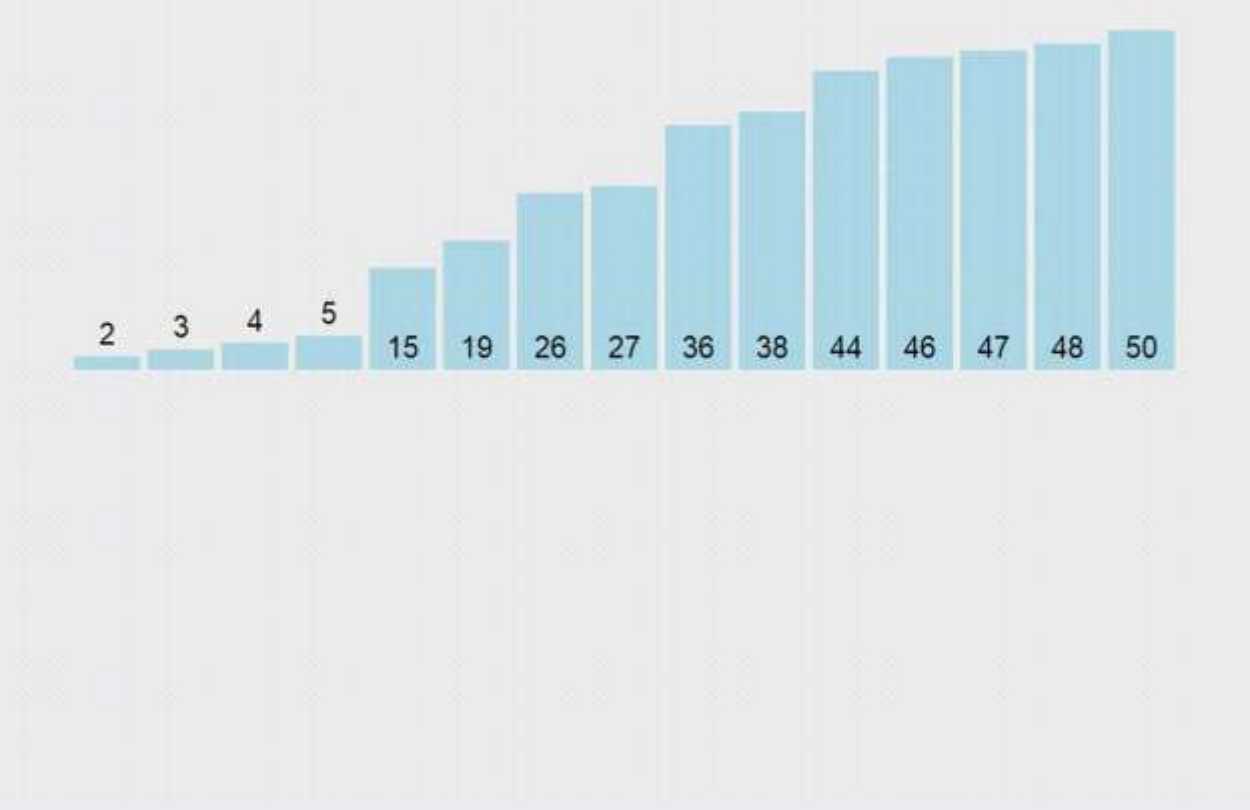
Популярные алгоритмы сортировки

Быстрая сортировка (Quick Sort). Один из самых эффективных алгоритмов. Как правило, выполняет сортировку быстрее остальных благодаря разделению массива на меньшие подмассивы и последующей рекурсивной сортировке.



Популярные алгоритмы сортировки

Сортировка слиянием (Merge Sort). Алгоритм, гарантированно обеспечивающий O(n log n) даже в худшем случае. Подходит для больших объемов данных и сохранения стабильности сортировки.




Алгоритмы сортировки
Быстрая сортировка
Самая быстрая на практике
Разделяет массив на подмассивы
Сортировка слиянием
Временная сложность O(n log n)
Стабильная сортировка
Пузырьковая сортировка
Временная сложность O(n²)
Простая реализация
Сортировка вставками
Временная сложность O(n²)
Эффективна для небольших массивов
Сортировка выбором
Временная сложность O(n²)
Простая реализация
Пирамидальная сортировка
Временная сложность O(n log n)
Низкие накладные расходы

Сложность алгоритма

Временная сложность определяет время выполнения алгоритма.

- O(n) — линейная сложность, количество операций увеличивается пропорционально размеру входных данных.
- O(n2) — квадратичная сложность, количество операций возрастает квадратично.
- O(n log n) — логарифмическая сложность, оптимальное время для большинства сортировок, включая Quick Sort и Merge Sort.

Сложность алгоритма

Пространственная сложность определяет, сколько дополнительной памяти требует алгоритм помимо самого массива. Чем меньше, тем лучше, особенно при работе с большими объемами данных.

Внутренние алгоритмы сортировки

Работа выполняется в оперативной памяти. Они отлично справляются с задачами, когда объем данных небольшой и все помещается в ОЗУ.

Внешние алгоритмы сортировки

Внешние алгоритмы сортировки подключают жесткий диск или другие внешние устройства хранения. Используются, когда данных так много, что они не помещаются в оперативной памяти. Например, многопутевая сортировка слиянием разбивает их на несколько файлов, а затем соединяет в один отсортированный массив.

Сортировка пузырьком

```
//метод сортировки пузырьком
int[] BubbleSort(int[] mas)
{
    int temp; //временная переменная
    //внешний цикл
    for (int i = 0; i < mas.Length; i++)
    {//внутренний цикл
        for (int j = i + 1; j < mas.Length; j++)
        {
            if (mas[i] > mas[j])
            {//меняем местами элементы в массиве
                temp = mas[i];
                mas[i] = mas[j];
                mas[j] = temp;
            }
        }
    }
    return mas;
}
```

Сортировка пузырьком

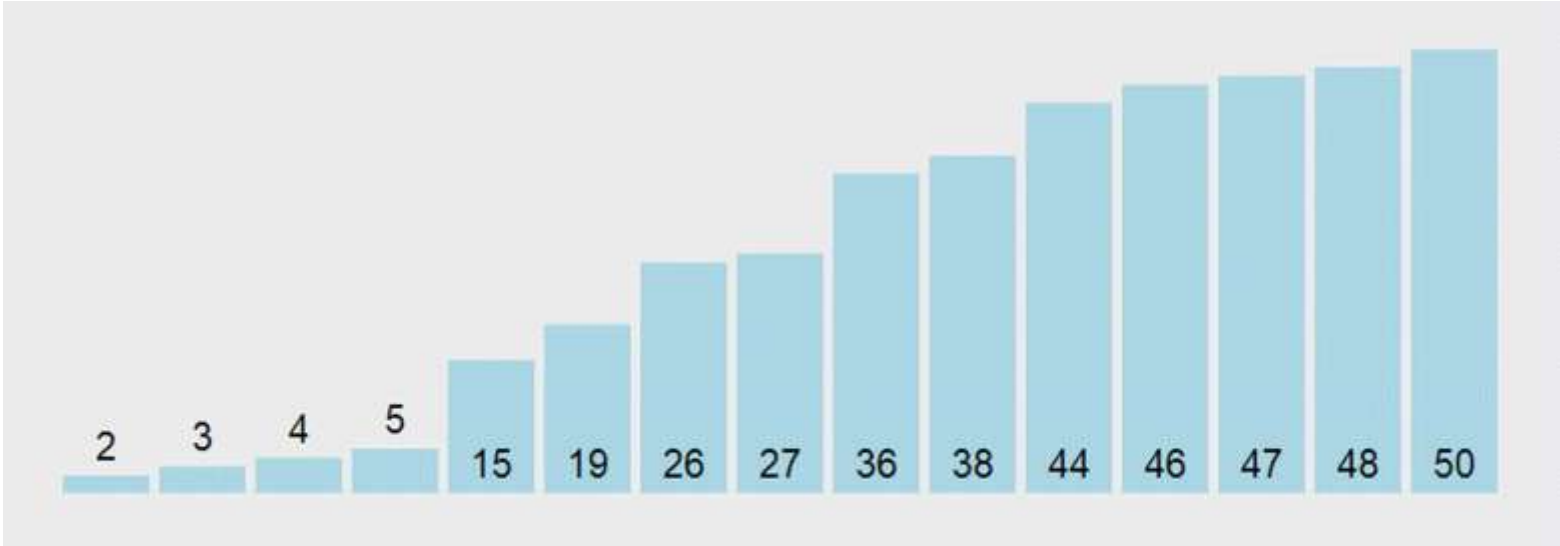
```
//изначальный массив
int[] arr = { -5, 3, 0, -22, 5, 7, 3, -1 };
//вызов метода для сортировки
arr = BubbleSort(arr);
//вывод массива на консоль
foreach (var item in arr)
{
    Console.WriteLine(item);
}
```

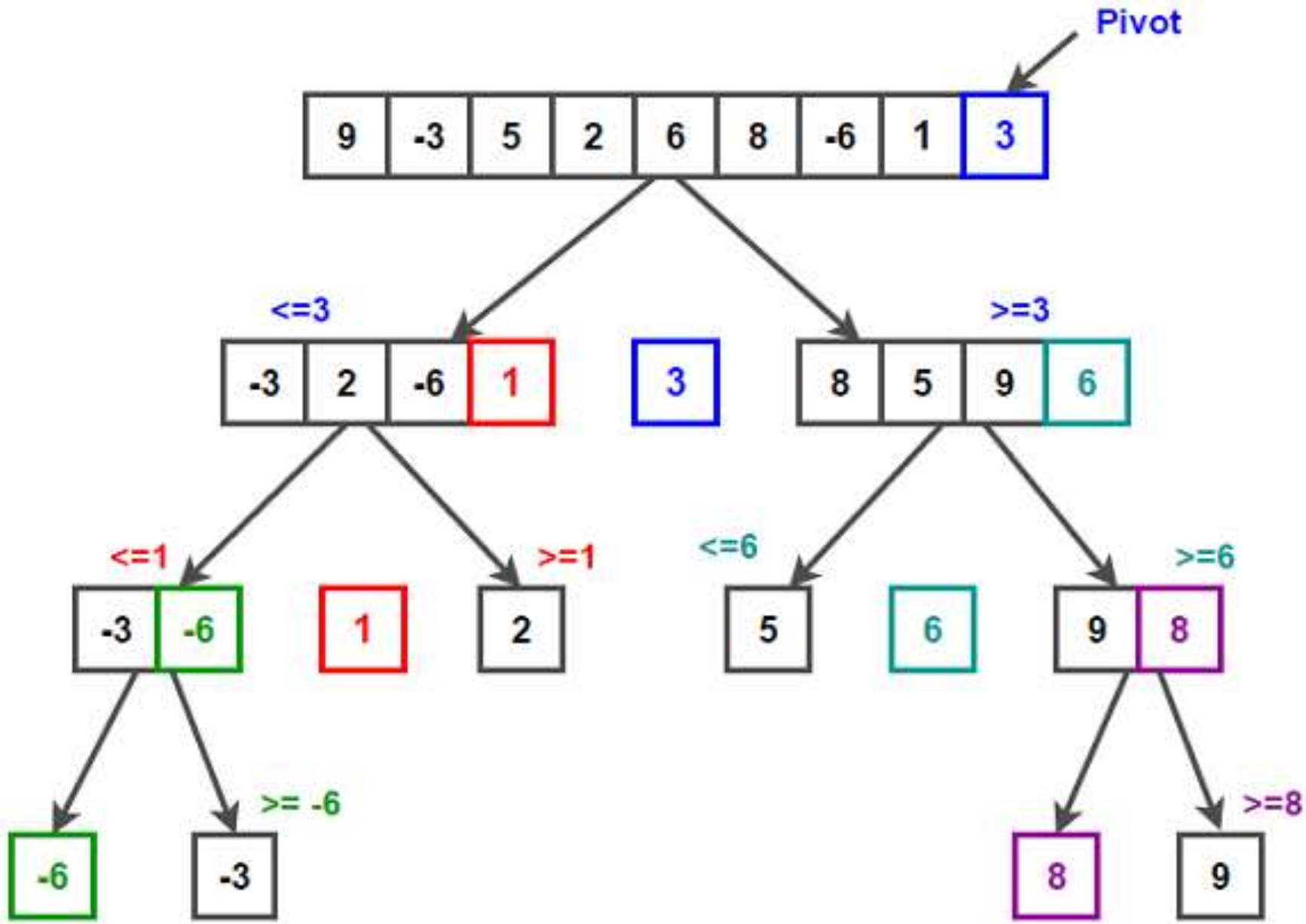
Сортировка выбором

```
 static int[] ViborSort(int[] mas)
{
    //внешний цикл
    for (int i = 0; i < mas.Length - 1; i++)
    {
        //поиск минимального числа
        int min = i;
        for (int j = i + 1; j < mas.Length; j++)//вложенный цикл
        {
            if (mas[j] < mas[min])
            {
                min = j;
            }
        }
        //обмен элементов
        int temp = mas[min];
        mas[min] = mas[i];
        mas[i] = temp;
    }
    return mas;
}
```

Быстрая сортировка, Quick Sort

Основная идея базируется на парадигме «разделяй и властвуй». Алгоритм сначала выбирает опорный элемент (pivot). Затем делит массив на два подмассива: в первом элементы меньше или равны pivot, а во втором — больше. После этого каждый подмассив сортируется независимо. Затем процесс рекурсивно повторяется для обоих подмассивов, что позволяет эффективно упорядочить элементы.




Pivot
9 -3 5 2 6 8 -6 1 3
<=3
>=3
-3 2 -6 1
3
8 5 9 6
<=1
>=1
<=6
>=6
-3 -6
1
2
5
6
9 8
>= -6
>=8
-6
-3
8
9

Быстрая сортировка

```
int[] QuickSort(int[] arr)
{
    if (arr.Length <= 1)
    {
        return arr;
    }

    int pivot = arr[arr.Length / 2];
    var left = new List<int>();
    var middle = new List<int>();
    var right = new List<int>();

    foreach (int x in arr)
    {
        if (x < pivot)
        {
            left.Add(x);
        }
        else if (x == pivot)
        {
            middle.Add(x);
        }
        else
        {
            right.Add(x);
        }
    }

    return [.. QuickSort(left.ToArray()), .. middle, .. QuickSort(right.ToArray())];
}
```

Задание

- Пользователь вводит массив с консоли
- Вывести массив отсортированный от большего к меньшему пузырьком.